HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO GIỮA KỲ

Xây dựng Chatbot hỏi đáp về lịch sử Việt Nam sử dụng GraphRAG

Lớp: E22HTTT

Tên: Nguyễn Đức Anh

ID: B22DCDT017

Người hướng dẫn: Kim Ngọc Bách

Hà Nội – 2026

Báo Cáo Tuần 5

Building a Neo4j-backed Chatbot using Python

1. System Overview

The system is an expert movie chatbot application that allows users to query information using natural language. The application combines an Agentic workflow and RAG (Retrieval-Augmented Generation) to retrieve data from a graph database (Neo4j).

Core Technologies:

- Frontend: Streamlit.
- Natural Language Processing (LLM) & RAG Framework: LangChain, OpenAI.
- Database: Neo4j (supporting both traditional Cypher queries and Vector Search).

2. Detailed Module Analysis

2.1. Configuration and Core Connections

- graph.py: Initializes a direct connection to the Neo4j database using the Neo4jGraph class. Credentials are fully secured via st.secrets.

```
import streamlit as st

# tag::graph[]
from langchain_neo4j import Neo4jGraph

graph = Neo4jGraph(
    url=st.secrets["NEO4J_URI"],
    username=st.secrets["NEO4J_USERNAME"],
    password=st.secrets["NEO4J_PASSWORD"],
    database=st.secrets["NEO4J_DATABASE"],
)
```

llm.py: Initializes the Large Language Model (LLM) used for reasoning and the Embeddings model to convert text into vectors for semantic search.

```python
import streamlit as st

# tag::llm[]
# Create the LLM
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    openai_api_key=st.secrets["OPENAI_API_KEY"],
    model=st.secrets["OPENAI_MODEL"],
)
# end::llm[]

# tag::embedding[]
# Create the Embedding model
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(
    openai_api_key=st.secrets["OPENAI_API_KEY"]
)
# end::embedding[]
```

2.2. Retrieval Tools

- vector.py (Semantic Search / RAG): Sets up a vector index to retrieve the plot property of the Movie node along with its metadata.

```python
from langchain_core.prompts import ChatPromptTemplate
# end::import_chat_prompt[]


# tag::vector[]
neo4jvector = Neo4jVector.from_existing_index(
    embeddings,                              # <1>
    graph=graph,                             # <2>
    index_name="moviePlots",                 # <3>
    node_label="Movie",                      # <4>
    text_node_property="plot",               # <5>
    embedding_node_property="plotEmbedding", # <6>
    retrieval_query="""
RETURN
    node.plot AS text,
    score,
    {
        title: node.title,
        directors: [ (person)-[:DIRECTED]->(node) | person.name ],
        actors: [ (person)-[r:ACTED_IN]->(node) | [person.name, r.role] ],
        tmdbId: node.tmdbId,
        source: 'https://www.themoviedb.org/movie/'+ node.tmdbId
    } AS metadata
"""
)
```

Then, it creates the get_movie_plot function combining the retriever and a RAG chain to answer questions based on the retrieved context:

```python
retriever = neo4jvector.as_retriever()
# end::retriever[]

# tag::prompt[]
instructions = (
    "Use the given context to answer the question."
    "If you don't know the answer, say you don't know."
    "Context: {context}"
)

prompt = ChatPromptTemplate.from_messages(
    [
        ("system", instructions),
        ("human", "{input}"),
    ]
)
# end::prompt[]

# tag::chain[]
question_answer_chain = create_stuff_documents_chain(llm, prompt)
plot_retriever = create_retrieval_chain(
    retriever,
    question_answer_chain
)
# end::chain[]

# tag::get_movie_plot[]
def get_movie_plot(input):
    return plot_retriever.invoke({"input": input})
# end::get_movie_plot[]
```

cypher.py (Text-to-Cypher): Uses GraphCypherQAChain to directly translate natural language questions into Cypher statements.

```
cypher_qa = GraphCypherQAChain.from_llm(
    llm,
    graph=graph,
    verbose=True,
    cypher_prompt=cypher_prompt,
    allow_dangerous_requests=True
)
```

2.3. Agent Coordination and Memory (ReAct Agent & Memory)

- Defining Tools for the Agent in agent.py: The agent is equipped with 3 tools: general chat, movie plot search, and graph relationship information search.

```
tools = [
    Tool.from_function(
        name="General Chat",
        description="For general movie chat not covered by other tools",
        func=movie_chat.invoke,
    ),
    Tool.from_function(
        name="Movie Plot Search",
        description="For when you need to find information about movies based on a plot",
        func=get_movie_plot,
    ),
    Tool.from_function(
        name="Movie information",
        description="Provide information about movies questions using Cypher",
        func = cypher_qa
    )
]
```

Initializing Graph Memory: Saves the chat history into Neo4j grouped by each user's session.

```
def get_memory(session_id):
    return Neo4jChatMessageHistory(session_id=session_id, graph=graph)
```

Initializing the Agent: Applies the ReAct (Reasoning and Acting) model combined with memory.

```python
agent = create_react_agent(llm, tools, agent_prompt)
agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    verbose=True
    )

chat_agent = RunnableWithMessageHistory(
    agent_executor,
    get_memory,
    input_messages_key="input",
    history_messages_key="chat_history",
)
```

2.4. User Interface and Execution (Frontend)

- UI Communication with utils.py: Helper functions to display messages on the Streamlit interface and retrieve the session ID to manage separate memory for each browser tab.

```python
# tag::write_message[]
def write_message(role, content, save = True):
    """
    This is a helper function that saves a message to the
     session state and then writes a message to the UI
    """
    # Append to session state
    if save:
        st.session_state.messages.append({"role": role, "content": content})

    # Write to UI
    with st.chat_message(role):
        st.markdown(content)
# end::write_message[]

# tag::get_session_id[]
def get_session_id():
    return get_script_run_ctx().session_id
# end::get_session_id[]
```

Starting the Agent (agent.py): The generate_response function is called along with the user's input and their session_id to execute the reasoning flow.

```python
def generate_response(user_input):
    """
    Create a handler that calls the Conversational agent
    and returns a response to be rendered in the UI
    """

    response = chat_agent.invoke(
        {"input": user_input},
        {"configurable": {"session_id": get_session_id()}},)

    return response['output']
```

Handling the Chat Interface (bot.py): The loop receives input from the chat bar, calls the submission handler, and displays a "Thinking..." spinner while waiting for the Agent's reply.

```python
def handle_submit(message):
    """
    Submit handler:

    You will modify this method to talk with an LLM and provide
    context using data from Neo4j.
    """

    # Handle the response
    with st.spinner('Thinking...'):
        # Call the agent
        response = generate_response(message)
        write_message('assistant', response)
```